

Received September 21, 2021, accepted October 23, 2021, date of publication October 26, 2021, date of current version November 3, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3123277

High-Resolution Waveform Capture Device on a Cyclone-V FPGA

NOELOIKEAU F. CHARLOT¹, DANIEL J. GAUTHIER¹, (Member, IEEE),
AND ANDREW POMERANCE²

¹Department of Physics, The Ohio State University, Columbus, OH 43202, USA

²Potomac Research LLC, Alexandria, VA 22311, USA

Corresponding author: Noeloikeau F. Charlot (charlot.5@osu.edu)

This work was supported by the Department of the Army under Award W31P4Q-20-C-0003. A preliminary patent was filed based on this work under Application Number 63/177,435 for “5-PS-Resolution Waveform-Capture-Device on a Cyclone V Field-Programmable Gate-Array With Dynamic Phase-Shifting.”

ABSTRACT We introduce the waveform capture device (WCD), a flexible measurement system capable of recording complex digital signals on trillionth-of-a-second (ps) time scales. The WCD is implemented via modular code on an off-the-shelf field-programmable gate-array (FPGA, Intel/Altera Cyclone V), and incorporates both time-to-digital converter (TDC) and digital storage oscilloscope (DSO) functionality. The device captures a waveform by taking snapshots of a signal as it propagates down an ultra-fast transmission line known as a carry chain (CC). It is calibrated via a novel dynamic phase-shifting (DPS) method that requires substantially less data and resources than the state-of-the-art. Using DPS, we find the measurement resolution - or mean propagation delay from one CC element to the next - to be 4.91 ± 0.04 ps (4.54 ± 0.02 ps) for a pulse of logic high (low). Similarly, we find the single-shot precision - or mean error on the timing of the waveform - to be 29.52 ps (27.14 ps) for pulses of logic high (low). We verify these findings by reproducing commercial oscilloscope measurements of asynchronous ring-oscillators on FPGAs, finding the mean pulse width to be 0.240 ± 0.002 ns per inverter gate. Finally, we present a careful analysis of design constraints, introduce a novel error correction algorithm, and sketch a simple extension to the analog domain. We also provide the Verilog code instantiating our design's hardware primitives in an Appendix, and make our FPGA interfacing methods available as an open-source Python library at <https://github.com/Noeloikeau/fpyga>.

INDEX TERMS Time-to-digital converter (TDC), digital storage oscilloscope (DSO), field programmable gate array (FPGA), phase lock loop (PLL), dynamic phase shift (DPS), carry chain (CC).

I. INTRODUCTION

Measuring time-dependent digital waveforms with accuracy and precision is crucial in many fields of science and engineering, with applications ranging from wireless communication systems and GPS to medical imaging and PET scans [1], [2]. Depending on the use-case, two technologies are routinely employed: time-to-digital converters (TDCs) [3], and digital storage oscilloscopes (DSOs) [4]. DSOs measure the history of a signal as it varies over time (its waveform), with analog-input (AI-DSO) and digital-input (DI-DSO) commonly available. TDCs measure the time interval between digital events, akin to a stopwatch. Historically, the two technologies have been composed of different application-specific integrated-circuits (ASICs).

The associate editor coordinating the review of this manuscript and approving it for publication was Christian Pilato¹.

Recently however, field-programmable gate arrays (FPGAs) have become an attractive alternative to ASICs for many applications. FPGAs can act as a ‘one-size fits all’ solution due to their ability to implement nearly any circuit by reconfiguring their hardware using only code and software.

Software-defined DI-DSOs based on standard FPGA logic elements are usually found in modern FPGA electronic design suites for debugging user-defined circuits. The sample rate of these tools is usually limited to the range of 200-300 MSamples/s because of the difficulties of registering the digital signal at a specific location in the circuit and storing the value in memory.

On the other hand, FPGA-based TDCs have flourished in recent years using existing FPGA resources in a non-standard configuration. One approach is to use the carry-chain (CC) elements built into each logic element to realize a high-speed transmission line for a digital signal. Here, it is assumed that

the pulse width is longer than the transmission line delay time and the goal is to register the edge of the pulse. The CC elements have been especially designed for high-speed operation of on-chip adders, and a digital signal can be passed from one CC element to the next on the picosecond time scale. Each CC element has an associated register, which can be triggered by a global clock signal to freeze the digital waveform contained in the transmission line. The digital output of the TDC is a word that encodes the time difference between the edge and the reference clock.

Using this approach, TDCs have been realized with few ps resolution [5]–[7]. However, calibrating the FPGA-TDCs is challenging because it is difficult to generate ps-scale signals on the chip. Also, signal distortions have to be corrected that arise from small timing variations of the signal from the carry-chain to the register, and from digital pulse contraction or expansion due to differences in the propagation speed of rising or falling edges along the transmission line [8].

FPGA-TDCs have accounted for their shortcomings with a variety of calibration procedures, the goal of which is to obtain accurate and precise measurements from raw data (*i.e.*, data that has not been post-processed). In the case of a TDC, the goal is to obtain a linear relation between the timing between an edge of an input digital signal and a reference clock edge. Many calibration methods have been proposed, such as statistical code density testing [3], wave-union [9], direct-to-histogram [10], multi-phase sampling [11], variable clock generation [12]–[14], and dynamic reconfiguration [15], [16].

Many of these calibration techniques require complex circuit architectures, high logic utilization, and large data sets. This is due to the aforementioned difficulty in generating ps-scale time intervals on FPGAs, which translates into a lack of control over the calibration signal, its time of application, or the measurement time. Clever solutions to this problem have been devised, such as the statistical code density test [3], which uses signals that are randomly distributed with respect to the reference clock. This method generates a uniform sampling of time intervals, but requires a large amount of data to obtain statistically relevant results. For these reasons, the calibration of an FPGA-TDC is often the most challenging and tedious part of the design. The result of the calibration process is a compressed, corrected, or otherwise encoded signal using auxiliary circuits such as edge detectors, thermometer-to-binary encoders, and bubble-error correctors [3], [17], [18].

The design of our waveform capture device (WCD) is similar to these FPGA-TDC designs, but it uses long CCs to capture a digital waveform without encoding it. As a result, we are able to directly measure the dynamics of arbitrary digital signals - not just timestamp pulse edges - with little overhead. We demonstrate both TDC and DI-DSO capabilities with precise measurements of complex GHz frequency waveforms entirely within an FPGA.

We calibrate our device using a novel method we refer to as dynamic phase-calibration (DPC). The method is applicable

to FPGA-TDCs starting with the Cyclone V and Xilinx 7 families of FPGAs, with continued support for the most recent chip families. DPC relies on a technology known as dynamic phase-shifting (DPS) [19], [20], which functions by adjusting the phase of a stable on-board clock known as a phase-lock loop (PLL). DPS reliably generates precise ps-scale time-intervals without re-configuring the FPGA. As a result, many of the former difficulties and complications associated with TDC calibration are eliminated.

As with any temporal measurement system, we are limited by the precision and accuracy of the reference clock used to trigger the waveform capture registers. Here, we use the DE10-Nano-SoC demonstration board manufactured by Terasic, which has a Siliconix phase locked loop model # Si5350C driven by a 27-MHz crystal oscillator. The Siliconix device has a typical period and cycle-to-cycle jitter of 40 ps and 50 ps, respectively, over 10,000 oscillations [21]. As we show, this jitter is the primary source of error in our measurement system, however the oscillator can easily be replaced by a higher-stability device as needed.

Our unique contributions are as follows:

- We introduce a novel FPGA-based measurement system using CCs that is capable of directly capturing digital waveforms on ps time-intervals, incorporating the core functionality of DI-DSOs and TDCs while using a simpler architecture.
- We introduce a novel calibration method using dynamic phase-shifting that is applicable to current DI-DSOs and TDCs on FPGAs, replacing the large logic overhead and data requirements of the state-of-the-art with a straightforward velocity calculation.
- We validate these claims through several case studies on PLLs and ring-oscillators, finding agreement with previous studies. We go on to report measurements of FPGA logic with unparalleled precision, study complex digital waveforms, and identify novel error correction techniques to reduce signal distortions in CCs on the Cyclone V.

The paper is organized as follows. In Sec. II, we describe the design and implementation of the WCD. In Sec. III, we examine the raw data and perform calibrations and error correction. In Sec. IV, we validate the design using Ring Oscillators as a case-study. We conclude in V with a discussion and future work.

All experiments are performed on DE10-Nano SOCs hosting Cyclone V 5CSEBA6U23I7 FPGAs using the Quartus Prime Lite version 20.1 CAD software. Data is collected over the JTAG interface using the open-source ‘fpyga’ Python package maintained by the primary author at <https://github.com/Noeloikeyau/fpyga>, which also interfaces with Quartus to implement all location and timing constraints. Our design uses low-level control over the FPGA primitives. These are poorly documented [22]; as such, we provide the Verilog code for these primitives in the Appendix.

II. WCD DESIGN AND OPERATION

In this section, we begin by summarizing the working principles and overall design of the WCD before presenting a careful analysis of the design constraints and implementation in the following subsection. In short, the WCD is a digital buffer that continuously measures the history of an input signal, and DPC is a velocity calculation that measures the distance travelled down this buffer after a phase-shift.

The core component of the WCD is a tapped delay line (TDL), described in Fig. 1. The TDL is composed of a CC used to buffer an input signal and registers used to capture its waveform on the edge of a clock. The CC is composed of a chain of K carry-elements (CEs) that pass the signal to the next CE after some time-delay τ_k . Each CE is composed of a redundant logic element such as an adder with fixed inputs whose sole purpose is to route the input signal down the special high-speed transmission lines that FPGAs reserve for carry operations. As in Fig. 1 (b), the signal also passes from each carry to a corresponding register. Each register stores the signal at that point in the chain as a bit x_k when a global clock C goes high, effectively capturing the waveform.

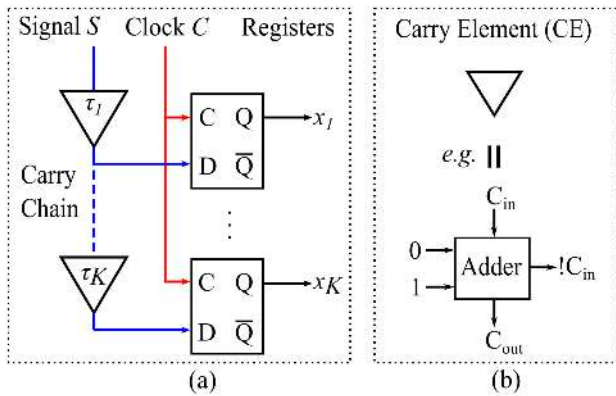


FIGURE 1. Primary building-blocks of the WCD. (a): Tapped delay line (TDL) composed of multiple registers and (b): Carry elements (CEs). Note: the CE is not truly an adder, see the appendix.

The concatenation of the register outputs forms a K -bit digital word $X = \{x_1, \dots, x_K\} \in \{0, 1\}^K$ directly encoding the captured waveform. Suppose each of the K carries acts as a nominal time-delay τ . It takes $K \times \tau$ time for S to travel down the CC. Further suppose that the clock C triggers all registers in the TDL simultaneously at time t . The TDL captures S backward in time from t in K increments of time τ . Hence, the principle of waveform capture is

$$X(t) = \theta(\{S(t - \tau), \dots, S(t - K \times \tau)\}), \quad (1)$$

where $\theta : [S_{min}, S_{max}]^K \rightarrow \{0, 1\}^K$ is an element-wise Booleanization operation representing the digitization of the signal performed by the registers.

Figure 2 shows the WCD and DPS-calibration block diagram. The DPS-PLL (dynamic phase-shifting phase-lock loop), RAM, and TDL are governed by control logic, and the entire WCD is clocked continuously by a 200 MHz PLL

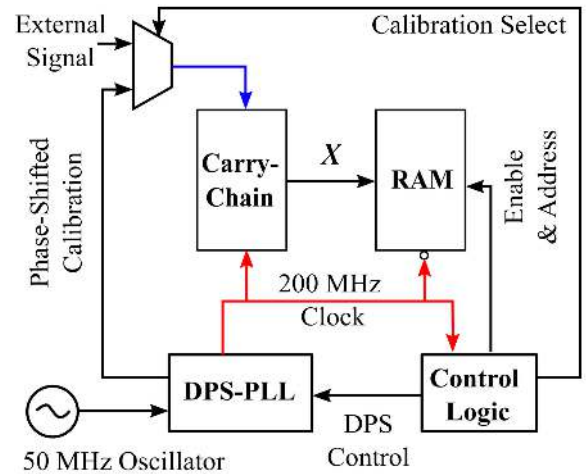


FIGURE 2. WCD block diagram.

output C . When not in calibration mode, the signal to capture the last $K \times \tau = (1300)(\approx 5 \text{ ps}) = 6.5 \text{ ns}$ of the waveform of S is sent to the TDL every $T = 5 \text{ ns}$ on the positive edge of C . This provides redundancy and ensures no portion of the waveform is lost. The data is then pushed to RAM as a $K = 1300$ bit digital word on the negative edge of C , chosen to avoid read/write conflicts with the TDL. This repeats until 512 periods or $2.56 \mu\text{s}$ have passed (unless otherwise stated), and the data is transferred onto a computer. We stress that there is no dead-time on the WCD when not calibrating, and the device acts as a DI-DSO / TDC hybrid.

When calibrating, the signal S is switched by the control logic to a second PLL output whose frequency is a ratio of 200 MHz. S is then shifted in phase relative to C in increments $\Delta t = 78 \text{ ps}$ by engaging the phase-shifting ports. When the DPS operation finishes after a minimum of two 50 MHz cycles of the external oscillator, the memory address is updated, the write enable signal is sent by the control logic to the RAM, and the TDL captures S on the rising edge of C . The net phase shift on the signal at this time is $n\Delta t$, where n is the memory address at which X is stored on the negative edge of C . The process repeats until 512 increments have occurred, corresponding to a phase-shift of 4π , and the difference in carry positions of the edge of the signal at each phase-shift are used to calibrate the device.

A. CONSTRAINTS AND IMPLEMENTATION

The WCD is fundamentally limited by the mean carry time τ . This is the smallest time-interval between which the signal is registered, and is often referred to as the resolution [23]. As we show, this value is approximately $\tau \sim 5 \text{ ps}$ for the Cyclone V architecture. The resulting design space is then constrained by the length of the carry chain K and the period of the capture clock T .

For the WCD to continuously measure S , the time required for the signal to travel down the chain $K \times \tau$ must be greater than or equal to the time between captures T . Otherwise,

portions of the waveform are lost in between clock intervals. Hence, we require $K \times \tau \geq T$. Further constraints on K arise due to the device-dependent properties of the FPGA architecture. For the Cyclone V DE10 Nano, the maximum CC length is $K = 1740$, which spans the height of the chip. Because the carry must enter from the top of the chain, this sets the maximum possible buffer time $K \times \tau$. As we show below, there also exist variations in the rise and fall times of the carry elements $\tau_{rise} > \tau_{fall}$. Consequently, a pulse traveling down the chain experiences contraction due to the falling edge (tail) catching up to the rising edge (head) of the pulse. This further limits K according to the tolerable level of pulse-shortening. That is, if the minimum time between pulses to be resolved is T_{min} , then $K \times (\tau_{rise} - \tau_{fall}) < T_{min}$. In practice we find $K \leq 1500$ prevents total degradation of sub-nanosecond pulses.

There are further constraints on measuring the signal S . If S is generated external to the FPGA, it must first pass through a pin and an I/O buffer before being routed onto the chip and measured. Within the chip fabric, S must also enter the TDL through an initial copy gate at the top of the CC before being carried, as it cannot be fed directly into a carry port. The copy gate and I/O buffer both have high bandwidth but introduce signal distortions such as a difference in rise and fall time. Most importantly, the path the signal takes through the CC must be set as a false path in the timing analyzer. This is because the travel time through the CC imposes false timing constraints that cannot be met by the compiler and must be ignored to avoid critical timing failures.

Timing constraints are primarily the skew - or difference in travel time of the clock signal - to each register, as well as the travel time required for the data to be transferred from the TDL to RAM. These are minimized by routing the on-chip PLL governing waveform capture through a global clock fabric minimizing the skew, as well as placing the RAM and TDL directly adjacent to each-other as shown in Fig. 3. We find that these techniques allow for continuously registering the CC without dead-time up to a 200 MHz PLL output, yielding a time between captures of $T = 5$ ns. This strikes a balance between the nominal minimum period $T = 1.43$ ns derived from the maximum frequency of 700 MHz for Cyclone V PLLs, and the approximate maximum buffer time of 7.5 ns due to a $K = 1500$ CC.

DPS calibration is limited by the minimum possible dynamic phase shift $\Delta t = 78$ ps for a Cyclone V, given by $1/8$ the maximum frequency of 1600 MHz for the voltage controlled oscillator (VCO) governing the PLL. This phase-shift must be minimized to maximally populate the number of edge positions recorded at each carry during calibration. Dead-time arises due to the process of engaging the auxiliary ports on the PLL enabling DPS. The first two ports need only be set once, and specify the desired PLL output to phase-shift and the direction to phase-shift; in this study these are set to static values so as to shift the calibration signal in the positive direction. However, to begin DPS, the last port must be asserted logic high for two clock cycles of the 50 MHz

free-running external crystal oscillator onto which the PLL locks. This process unlocks the PLL to the reference oscillator and the re-locking procedure is asynchronous and device-dependent; in practice, we find it ranges from the minimum of two clock cycles to several dozen depending on the FPGA and operating conditions. For further details on engaging DPS in the Cyclone V and Xilinx 7, consult Ref. [19] and [20], respectively.

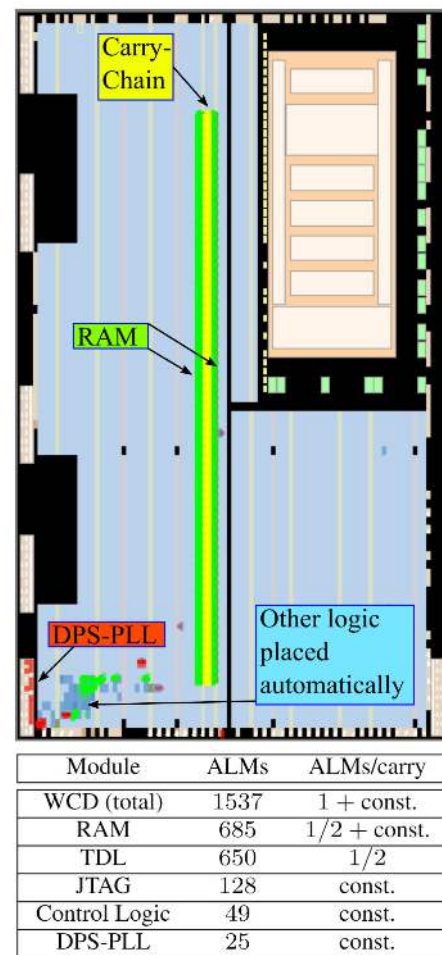


FIGURE 3. Resource allocation for the $K = 1300$ carry-element WCD on a cyclone V used in this study. Highlighted are the primary modules including the carry-chain/TDL, RAM, and the DPS/PLL.

Finally, the uncertainty on the timing of the waveform is limited by the relative timing errors on the two outputs of the PLL, and by the jitter of the Siliconix crystal oscillator that drives them. The Cyclone V reference manual lists the maximum jitter on the PLL over 15 standard deviations as 300 ps and on the phase shift as 50 ps [19], though these values are highly conservative. As we show in the next section, the actual timing error - quantified by the single-shot precision, or the root-mean-square timing error over the full CC [23] - is essentially equal to the nominal 30 ps jitter on the crystal oscillator. As a result, we identify this jitter on the crystal oscillator as the largest source of error in the WCD after all

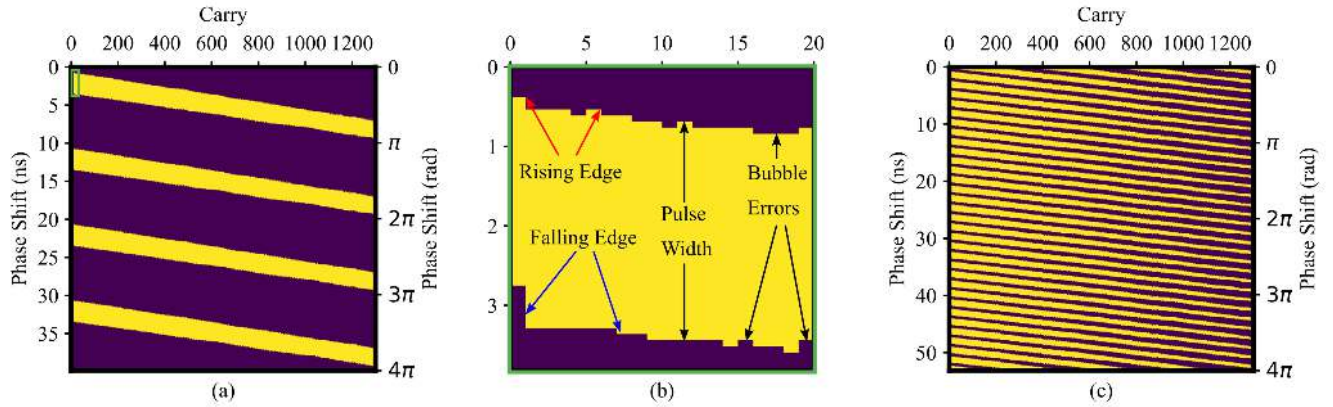


FIGURE 4. Raw data captured by the WCD from measurements of phase-shifted PLL outputs used for calibration. (a): 2.5 ns pulses, 78 ps / phase shift. (b): Zoom of green region in (a). (c): 0.83 ns pulses, 104 ps / phase shift.

other design constraints have been accounted for. Therefore, future WCD designs can obtain a greater precision by using a higher quality crystal oscillator with lower jitter.

III. CALIBRATION

In this section, we begin by examining the features of the DPS-calibration data in Fig. 4. We go on to describe the results and technique of the calibration in Fig. 5. Finally we introduce a novel error-correction algorithm in Fig. 6.

Figure 4 (a) and (c) show datasets obtained from dynamic phase-shifted calibration measurements of two different PLL outputs with different frequencies. Horizontal slices correspond to registered states of the chain X after the phase-shift indicated on the vertical direction. Yellow pixels are positions in the CC where the register latched 1, and dark purple 0. Note the unprecedented nature of the data: we capture waveforms at 10^{-12} s intervals without error correction or post-processing ('raw data'), from which individual features can readily be examined as in (b).

Shown in Fig. 4 (a) are 2.5 ns wide pulses generated every 10 ns by a 100 MHz, 25% duty-cycle PLL output. Every integer multiple of 2π , these pulses align with the rising edge of the 200 MHz capture clock. Hence, as the phase is incremented by $\Delta t = 78$ ps down the vertical, the pulses progress horizontally down the chain. The duty cycle is such that a single pulse exists within the carry chain at any given time. This simplifies the calculation of the speed of the rising and falling edges, for which a unique pair of tuples (phase shift, edge carry index) can be obtained from each horizontal slice. From this data, we calculate the speed of the edges and obtain a transfer function $t(x)$, which maps every carry position to a time interval relative to the capture event.

Shown in Fig. 4 (c) are 0.83 ns pulses occurring every 1.67 ns arising from a 600 MHz, 50% duty-cycle PLL output. This frequency and duty cycle ensures that multiple pulses exist within the CC at any given time, which simplifies a calculation of the rate at which pulse widths shrink as a function of the position of their falling edge. This phenomenon,

known as pulse shrinking, arises due to difference between the speed of the rising and falling edges, and can be seen by observing the pulse widths decrease within a single horizontal slice of (c). For this analysis, we use $\Delta t = 104$ ps due to limitations generating incommensurate PLL outputs at high frequencies.

As can be seen in Fig. 4 (b), the signal expands as it enters the chain. This is analogous to the change in propagation speed of a wave at the boundary between two media: the first element in the CC is a copy gate, which takes much longer to operate than the subsequent carry elements. This means that the largest differential nonlinearity, or deviation from a constant speed, occurs at the start of the chain. Additionally, bubble errors, which are due to jitter in the arrival of the clock signal at the registers (*i.e.*, not all registers latching simultaneously), are clearly visible. These non-idealities - pulse-shrinking, bubble errors, and differential nonlinearities - induce signal distortions.

However, we find that the signal remains faithfully represented even in the presence of these distortions. By way of explanation, we highlight several checks, some of which can be seen from Fig. 4. The first such check is to note that the nominal 2.5 ns width of the pulses in (a) are accurately represented along the vertical, demonstrating the stability of the phase-shift. This is also demonstrated by noting that the nominal 10 ns between pulses are clearly visible. Another check (not shown here) is to feed in a signal of constant logic high and ensure that the data are all 1's. This ensures the stability of the registers at the capture frequency.

We find that the rising and falling edges exhibit a highly linear relationship between edge position and phase shift so that carry rise and fall times can be calculated from a linear fit to the data. This is shown by the black dotted lines in Fig. 5 (a), in which we extract the (phase shift $n\Delta t$, edge carry index x_i) tuples from data of the kind in Fig. 4 (a) for the rising (red) and falling (blue) edges. This data is binned over eight TDL locations and three FPGAs, from which we obtain a distribution of carry indices at each phase shift, having

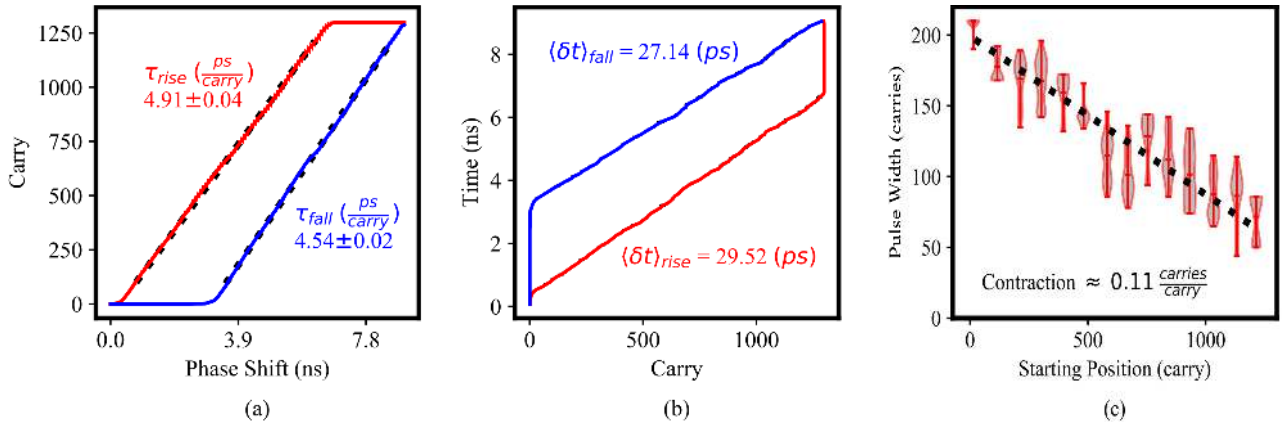


FIGURE 5. (a): Carry speed calibration over 8 TDLs on 3 FPGAs. (b): Transfer function for a single FPGA and TDL. (c): Pulse widths and linear contraction rate.

standard deviations $\delta x_i \sim 10$ carries (shown but not visible). These data with errors are fit to straight lines, the reciprocal slopes of which yield the quoted carry times for MLABs $\tau_{rise} = 4.91 \pm 0.04$ ps and $\tau_{fall} = 4.54 \pm 0.02$ ps. For LABs (not shown) we find 6.35 ± 0.15 ps, 5.60 ± 0.12 ps.

The data in Fig. 5 (a) can be transposed in order to obtain a mapping from carry index x to time interval $t(x)$, realizing full TDC functionality. In the TDC literature, a single TDL would be referred to as a channel and calibrated individually. This is shown in Fig. 5 (b). The uncertainty in the timing δt at carry index x is given by the local slope of the mapping $t(x)$ multiplied by the uncertainty in the edge carry index at that time, *i.e.*, $\delta t(x) = t'(x)\delta x$, where t' is the derivative of $t(x)$. The quoted uncertainties $\langle \delta t \rangle \sim 30$ ps are the RMS errors (single-shot precision) for each edge over the full dynamic range ($K \times \tau$). They are of the same magnitude as the jitter of the crystal oscillator [21], suggesting the primary source of error is this jitter.

The differences between rise time and fall time cause pulse shrinking, which we calculate explicitly in Fig. 5 (c) from the data shown in 4 (c). We define a pulse width as the length of any observed number of contiguous 1's. This definition means that bubble errors dominate the distribution and interrupt 'real' pulses. This is therefore a simple approximation, and we filter widths below 15 carries to exclude these bubbles. We bin these widths as a function of the position of their falling edge, and show the distributions as violin plots in red (only every 25th plot is shown for clarity). The resulting data is then fit to the dotted line, the absolute value of the slope being the quoted contraction. Finally, we observe no 'ultra-wide' bins [8] that have been observed in older FPGA process technologies - with the potential exception of the middle of the chain, which may be due to crossing clock-domains.

The fractional difference in rise and fall times is $1 - 4.91/4.54 \approx 8\%$, consistent with the 11% fractional rate of contraction obtained from this basic analysis (for LABs we find closer to 20%). This means that the rate of pulse-shrinking is essentially linear. As a consequence, this

type of signal distortion can be inverted with a simple error correction algorithm, such as dilating the pulses according to the observed rate of contraction. We examine such an algorithm in the remainder of this section.

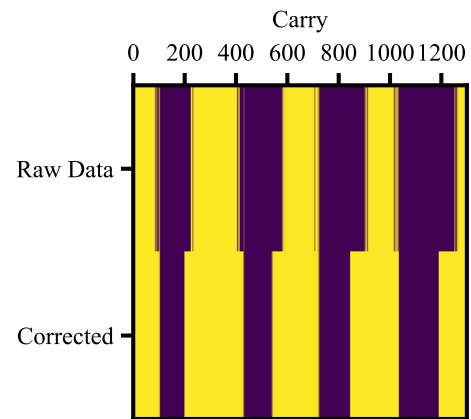


FIGURE 6. Raw and corrected data from a single state of the CC in 4 (c).

Figure 6 shows the result of applying error correction to a single state of the CC from Fig. 4(c). The algorithm fills in all bubble errors and reverses the effects of pulse contraction in software. We note that the copy element at the start of the CC acts as a low-pass filter that limits the switching time to ~ 50 ps; thus we isolate all contiguous regions of zeros of width greater than 15 carries and obtain the indices of these regions. For example, in the raw data shown in Fig. 6, the last such set of zeros is from carry 1033 to 1249. Next, we reverse pulse shortening by reducing the right endpoint of each empty region by a multiplicative factor - 0.95 in the image above (*e.g.*, $1249 \rightarrow \lfloor 0.95 \times 1249 \rfloor$). Finally, we reverse bubble errors by filling in all points which do *not* lie in these regions with logic high.

We stress that previous TDC error-correction algorithms rely on thermometer encodings, which are not applicable to arbitrary waveforms. However, our algorithm does not

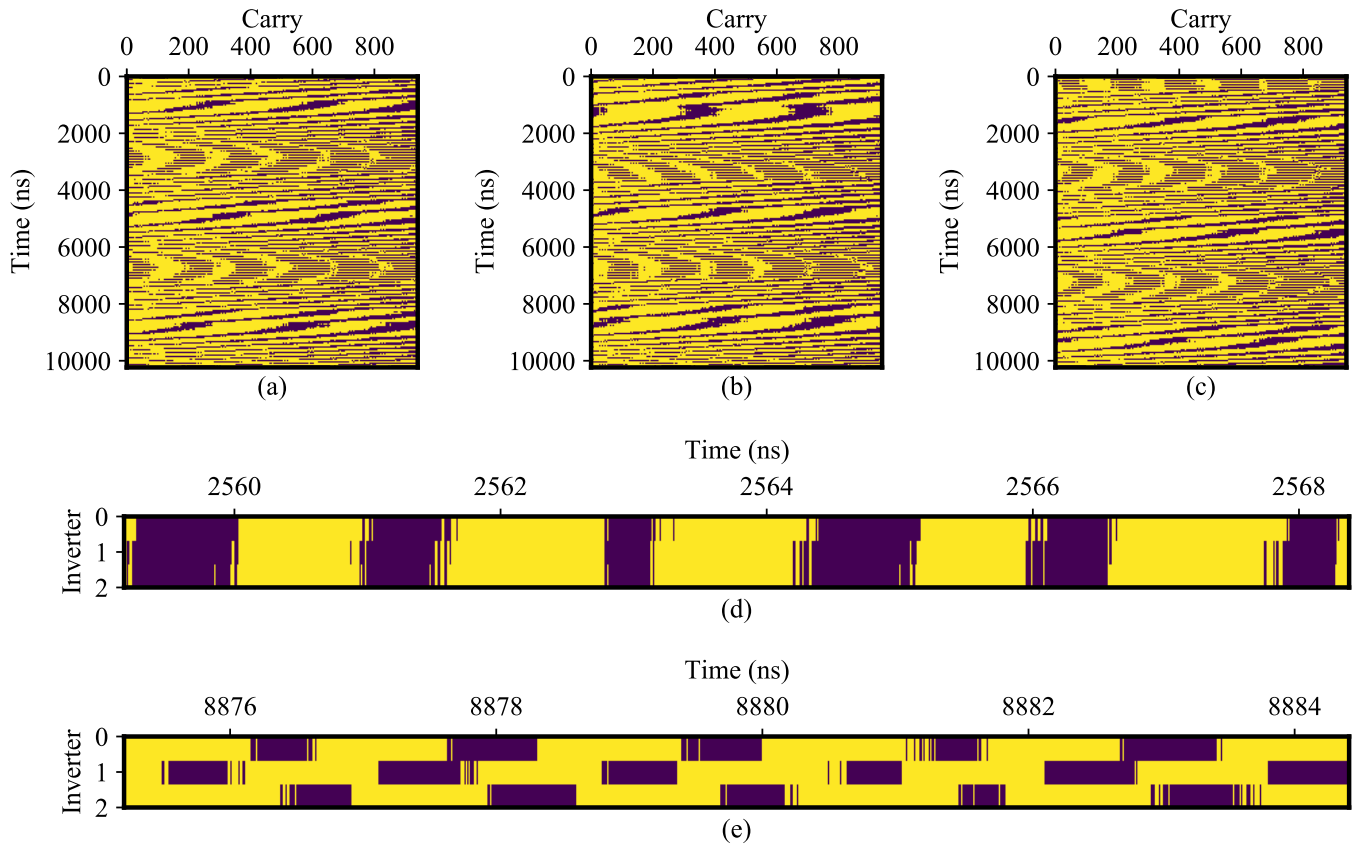


FIGURE 7. (a)-(c): Continuous capture of 3-node ring-oscillator initialized in the unstable (0,0,0) state. Zoom in of (d): in-phase and (e): out-of-phase oscillations.

account for the expansion at the start of the chain and the extra contraction in the center. More advanced algorithms correcting for these nonlinearities - such as functional transforms rather than constant multiplicative factors - are the subject of future work.

IV. CASE STUDY: ASYNCHRONOUS RING-OSCILLATOR

Here we briefly demonstrate the DSO capability of the WCD by continuously measuring an asynchronous (unclocked) system of logic gates known as a ring oscillator (RO). ROs are ubiquitous in hardware design and are a natural case study for the WCD. As we will show, our results match those from the literature and allow for precision study of FPGA logic with applications in, *e.g.*, novel attacks on FPGA-based cryptographic primitives.

ROs consist of inverter gates (or nodes) arranged in a unidirectional circle. Each node inverts the state of the previous node in the ring. When the logic gates are in a steady-state, pulses of logic-high travel synchronously around the ring [24]. However, when the states are unsteady, such as in an unclocked system, inversions occur as quickly as the hardware allows. These asynchronous dynamics exhibit truly analog behavior due to collisions between electronic pulses generating intermediary logic values. In these cases, the dynamics of the ring are highly complex and exhibit multi-timescale transients [25].

We capture these dynamics in Fig. 7. Shown in (a),(b),(c) are the continuously captured states of each individual node in a 3-node RO on a single FPGA. The transient and steady behavior discussed above is shown in (d) and (e). The initial state of the RO is set with a multiplexer to the unstable (0,0,0) state before being released and allowed to evolve in time asynchronously. We capture each node every $T = 5$ ns, and evaluate the transfer function to truncate the TDL after $T = 5$ ns (corresponding to $K = 940$) so that we can concatenate the images into a continuous time-series as in (d) and (e).

The observations of these transients reproduce those reported in Ref. [25], which used a commercial DSO to study the dynamics of a 3-node RO on a Cyclone IV FPGA that were routed off-chip. The steady behavior of (e) matches the theoretical expectations from [24]. Moreover, we find that the mean pulse width is 0.240 ± 0.002 ns per inverter gate (see Appendix for method). This is similar to the Cyclone IV 3-node RO inverter gate timescale of 0.28 ± 0.01 ns obtained by [26] using a commercial DSO. These results confirm the DSO capability of the WCD, as it is expected that the Cyclone V architecture used here has a faster operation time.

As future research, we will perform similar measurements to directly study the properties of other internal FPGA circuitry such as routing delays, PLL jitter, and the rise and fall time of individual logic elements. Such measurements have important consequences for FPGA-based cryptographic

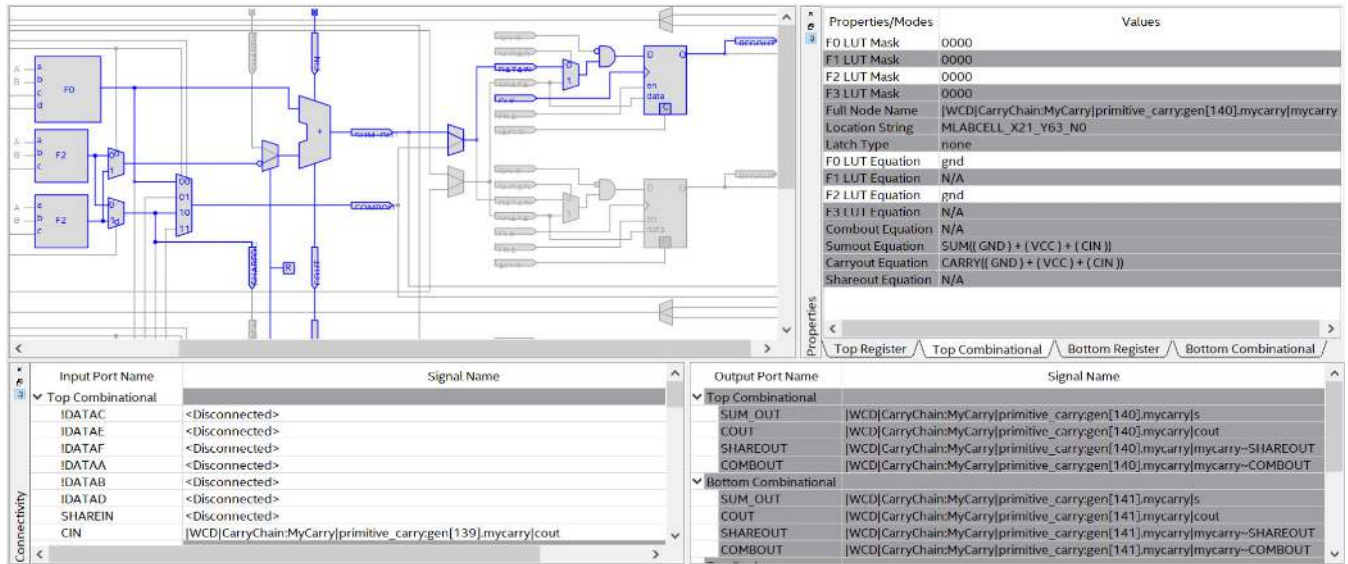


FIGURE 8. Altera Cyclone V low-level primitive implementation of Fig. 1, showing a typical carry element in a MLABCELL and associated register. Not shown: a special first MLABCELL, occurring at the top of the MLAB block, routes the input signal through a copy element, where it is carried down the chain. Subsequent LEs use a LUT mask of all 0s to eliminate redundant logic and explicitly carry the signal.

primitives such as those described in Ref. [27], which rely on obfuscation of these properties. As such, the WCD may open new attacks on these security architectures.

V. CONCLUSION

In summary, we use carry chains and phase-lock loops for the direct capture of digital waveforms and for calibration using dynamic phase-shifting. In doing so, we construct the first ps-scale-resolution FPGA-based measurement system with hybrid digital storage oscilloscope and time-to-digital converter functionality. As a result, the WCD finds application in any scenario where high-precision, ultra-fast measurements are required and a low-cost, off-the-shelf solution is desirable.

There are several advantages to WCD design and DPC. First, both are straightforward: fundamentally the WCD is a buffer, and DPC is a velocity calculation that requires only a single memory read-out. These concepts are easily accessible and, when implemented with the proposed architecture, yield raw-data that is both interpretable and precise. As a result, the WCD is natural for FPGA applications, which aim for ease-of-use.

In contrast, consider the calibrations and requirements of other TDC architectures: bin-realignment, statistical tests, coding-decoding, sorting algorithms, individual carry calibrations, *etc.* [3], [5]–[7], [9]–[11], [17], [28]–[30]. In Ref. [5], for example (a Cyclone V FPGA TDC), the raw data are randomly distributed and indiscernible - requiring an external hard-processor to implement a sorting algorithm on the encoded distribution of hundreds of randomly aligned clocks.

Second, as an FPGA-based measurement system the WCD design is flexible, and it has the advantage that it can measure signals generated internal to the FPGA without the need for

additional hardware. Additionally, due to its modular nature and similarity to DSOs and TDCs, integrated systems can easily incorporate the WCD in place of or in combination with these designs, and leverage existing error correction architectures such as bubble error correctors [18].

The flexibility of the design immediately presents several extensions of the WCD. For example, the sampling rate and resource count can be improved by using two registers per carry. DPS can be used to delay the measurement clock and incrementally capture repeated signals in Δt intervals. Data transfer off the board can be streamlined using the hard-processor rather than the JTAG interface.

Finally, we stress that analog waveform capture is also possible. Each TDL acts as a 1-bit ADC or comparator as described by θ in Eq. 1. Similar to a flash-ADC, the domain of the input waveform $[V_{min}, V_{max}]$ can be partitioned into a series of subsets ranging over the amplitude of the signal $[V_{min}, V_1), [V_1, V_2), \dots, [V_M, V_{max}]$ using external comparators having thresholds corresponding to each sub-interval. The output of each comparator can then be passed into a corresponding TDL. Each of the TDLs would thus trace the associated portion of the analog signal within that sub-domain. The continuous approximation of the amplitude of the waveform could then be recovered from each of the TDLs, yielding an analog WCD. Extending the WCD in this manner (for instance, using Field-Programmable Analog Arrays or FPAAs) is the subject of future work.

APPENDIX

In these appendices, we discuss how to implement the WCD on a generic FPGA using low-level primitives, how to apply design constraints to these primitives, and how to incorporate the WCD in other systems and applications. We then give the

code and details for the Cyclone V carry primitives and ring oscillators used in this study.

A. SPECIFYING PRIMITIVES AND CONSTRAINTS

For Intel devices, the primitive definitions for each FPGA can be found in the Quartus installation directory under the ‘\quartus\eda\sim_lib’ folder. Each primitive is defined within the corresponding chip and language file - here, we use the ‘cyclonev_lcell_comb’ and ‘cyclonev_ff’ modules defined in the ‘cyclonev_atoms.v’ file. For FPGAs other than the Cyclone V, these primitive declarations in the ‘primitive_carry’ module below must be altered to reflect the new definitions. Similarly, if the dynamic phase shifting process is different (for example, on Xilinx devices), then the algorithm described in Sec. II-A must be changed accordingly. However, for Intel devices with reconfigurable PLLs this is unnecessary due to the standardization of PLL megafunctions.

Timing and location constraints need to be specified for each carry element as discussed in Sec. II-A. These are dependent on the name of the logic in the user’s code and on the type of FPGA. For Intel devices, the Tcl command follows the syntax ‘set_false_path -from ___ -to ___’ where ‘___’ are names of successive carry elements. Similarly, location constraints must be specified to choose the location of the CC, to place capture registers next to carry elements, and to place carry elements next to RAM. This follows syntax of the form ‘set_location_assignment MLABCELL_Xx_Yy_Nn -to ___’ where ‘___’ is the logic to be placed at the corresponding xyn values. For LABs, we replace MLABCELL with LABCELL. Both of these constraints can be implemented by hand in the Quartus Timing Analyzer and Chip Planner GUIs, as well as using the fpyga Python package.

As discussed in more detail below, the look-up-table (LUT) mask for each carry primitive is set to all 0’s to eliminate redundant logic and force the signal to be routed directly from one carry port to the next. For other devices, an equivalent mask forcing the carry-in to be routed to carry-out must be used. Further modifying the LUT, for example to implement additional logic functions, is not advised, as this could introduce additional delays and in the worst case cause the signal to not be routed down the high-speed carry fabric.

Instead, operating downstream of the registers that capture the signal is a modular and scalable approach to using the WCD in other applications, and has no impact on performance. By operating on the registers adjacent to each carry element, rather than the carry elements themselves, one does not impact the waveform capture process as the signal has already been stored. These register primitives are specified below in the ‘primitive_ff’ module.

B. LOW-LEVEL PRIMITIVES ON CYCLONE V

The Verilog code used to instantiate our design is listed below and the result of its compilation using the Quartus CAD software is shown in Fig. 8. In a Cyclone V FPGA, the TDLs are instantiated using the low-level Altera Cyclone V primi-

tives [22]. Two of these primitives fit in each cell of a logic-array block (LAB) or memory logic-array block (MLAB). Each cell contains several look-up-tables (LUTs: F0, F1, F2, F3), multiplexers (MUXs), and registers (REGs). A hash specifies the logic function implemented by manipulating the LUTs, which cause the MPXs to route the input carry CIN through an adder. The adder simply passes the signal to the next MLABCELL and a register. The routing on the register ports are then controlled explicitly and the placement of all logic in each MLAB is set using location constraints.

For the experiments in Sec. IV, we place the RO compactly within a single MLAB and control the initial state of the RO using a multiplexer following the appendix of Ref. [27]. This yields Verilog code of the form ‘ring[i] = enable ? ring[i-1]: init[i]’ for nodes $i \bmod N$. For Fig. 7, we use nearby MLABs to create TDLs for each of the $N = 3$ nodes. To determine the pulse width per inverter gate, we instantiate an $N = 19$ node RO initialized to the stable state in which the first node is 1 and the rest are 0, and tap only the first node with a CC. We bin the distribution of pulse widths after filtering pulses of width below 15 carries to remove bubbles as in our analysis of pulse shrinking. We calculate the average width and standard deviation, and multiply by the carry time and propagate the

```

2 //This is the primitive carry element from Fig. 1
3 module primitive_carry (
4     input wire a,
5     input wire b,
6     input wire cin,
7     output wire cout,
8     output wire s
9 );
10
11 cyclonev_lcell_comb #(
12     .dont_touch("off") //default intel value
13 ) mycarry (
14     .dataa(),
15     .datab(b),
16     .datac(a),
17     .datad(),
18     .datae(),
19     .dataf(),
20     .datag(),
21     .cin(cin),
22     .sharein(),
23     .combout(),
24     .sumout(s),
25     .cout(cout)
26 );
27
28 //This LUT mask propagates the carry only
29 //It means that no logical function occurs
30 defparam mycarry.lut_mask=64'h0000000000000000;
31
32 endmodule
33
34
35
36 //This is the basic register primitive in Fig. 1
37 module primitive_ff (
38     input wire s,
39     input wire clk,
40     input wire clr,
41     input wire ena,
42     output wire q
43 );
44

```

```

45 cyclonev_ff myff (
46     .d(s),
47     .clk(clk),
48     .clrn(),
49     .aload(),
50     .sclr(clr),
51     .sload(),
52     .asdata(),
53     .ena(ena),
54     .devclrn(),
55     .devpor(),
56     .q(q)
57 );
58
59 endmodule
60
61 //This is the first input copy gate,
62 //as you cannot feed to carry port directly
63 module primitive_carry_in(
64     input wire datac,
65     output wire cout
66 );
67
68 cyclonev_lcell_comb #(
69     .dont_touch("off")
70 ) mycarryin (
71     .dataa(),
72     .datab(),
73     .datac(datac),
74     .datad(),
75     .datae(),
76     .dataf(),
77     .datag(),
78     .cin(0),
79     .sharein(),
80     .combout(),
81     .sumout(),
82     .cout(cout)
83 );
84
85 //This LUT mask passes the datac port to cout
86 defparam mycarryin.lut_mask=64'h00000000F0F0F0F;
87
88 //the mask works by concatenating the MPX LUTs
89 //these are f0,f1,f2,f3, and the mask has values
90 //mask[15:0]=f0; mask[31:16]=f1;
91 //mask[47:32]=f2; mask[63:48]=f3;
92 endmodule
93
94 //This is the carry chain itself
95 module carry_chain #(parameter N=256) (
96     input wire [N-1:0] a,
97     input wire [N-1:0] b,
98     input wire carryin,
99     input wire clk,
100     input wire enable,
101     input wire clear,
102     output wire [N-1:0] regout,
103     output wire carryout
104 );
105
106 //internal wire for passing carries
107 //one bit larger for initial combinatorial cell
108 //which passes data_in along carry chain
109 wire [N:0] c_internal;
110 //assign first input manually
111 primitive_carry_in mycarryin (
112     .datac(carryin),
113     .cout(c_internal[0]),
114 );
115 //internal wire for sum outs
116 wire [N-1:0] s;
117 //and rest with loop
118 genvar i;

```

```

119 generate
120 for (i=1; i<N+1; i=i+1) begin : gen
121
122     primitive_carry mycarry (
123         .a(a[i-1]),
124         .b(b[i-1]),
125         .cin(c_internal[i-1]),
126         .cout(c_internal[i]),
127         .s(s[i-1])
128     );
129
130     primitive_ff myff(
131         .s(s[i-1]),
132         .clk(clk),
133         .q(regout[i-1]),
134         .clr(clear),
135         .ena(enable)
136     );
137
138 end
139 endgenerate
140 assign carryout=c_internal[N];
141 endmodule

```

REFERENCES

- [1] X. Qin, W.-Z. Zhang, L. Wang, Y. Tong, H. Yang, Y. Rui, X. Rong, and J.-F. Du, "A pico-second resolution arbitrary timing generator based on time folding and time interpolating," *Rev. Sci. Instrum.*, vol. 89, no. 7, Jul. 2018, Art. no. 074701, doi: 10.1063/1.5037841.
- [2] W. Gao, D. Gao, C. Hu-Guo, and Y. Hu, "Integrated high-resolution multi-channel time-to-digital converters (TDCs) for PET imaging," in *Biomedical Engineering, Trends in Electronics, Communications and Software*, A. Laskovski, Ed. Rijeka, Croatia: InTech, Jan. 2011.
- [3] S. Tancock, E. Arabul, and N. Dahnoun, "A review of new time-to-digital conversion techniques," *IEEE Trans. Instrum. Meas.*, vol. 68, no. 10, pp. 3406–3417, Oct. 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8809777>
- [4] S. K. Pal, A. Kumar, and K. Kumawat, "Design and VLSI implementation of a digital oscilloscope," in *Proc. 4th Int. Conf. Comput. Intell. Commun. New.*, Mathura, India, Nov. 2012, pp. 473–476. [Online]. Available: <http://ieeexplore.ieee.org/document/6375158/>
- [5] T. Sui, "A 2.3-ps RMS resolution time-to-digital converter implemented in a low-cost cyclone V FPGA," *IEEE Trans. Instrum. Meas.*, vol. 68, no. 10, pp. 3647–3660, Oct. 2019.
- [6] Y. Wang and C. Liu, "A 4.2 ps time-interval RMS resolution time-to-digital converter using a bin decimation method in an UltraScale FPGA," *IEEE Trans. Nucl. Sci.*, vol. 63, no. 5, pp. 2632–2638, Oct. 2016. [Online]. Available: <http://ieeexplore.ieee.org/document/7562355/>
- [7] X. Qin, M.-D. Zhu, W.-Z. Zhang, Y.-H. Lin, Y. Rui, X. Rong, and J. Du, "A high resolution time-to-digital-converter based on a carry-chain and DSP48E1 adders in a 28-nm field-programmable-gate-array," *Rev. Sci. Instrum.*, vol. 91, no. 2, Feb. 2020, Art. no. 024708, doi: 10.1063/1.5141391.
- [8] J. Wu, "Several key issues on implementing delay line based TDCs using FPGAs," *IEEE Trans. Nucl. Sci.*, vol. 57, no. 3, pp. 1543–1548, Jun. 2010. [Online]. Available: <http://ieeexplore.ieee.org/document/5485143/>
- [9] J. Wu, "An FPGA wave union TDC for time-of-flight applications," in *Proc. IEEE Nucl. Sci. Symp. Conf. Rec. (NSS/MIC)*, Orlando, FL, USA, Oct. 2009, pp. 299–304. [Online]. Available: <http://ieeexplore.ieee.org/document/5401738/>
- [10] N. Dutton, J. Vergote, S. Gnechhi, L. Grant, D. Lee, S. Pellegrini, B. Rae, and R. Henderson, "Multiple-event direct to histogram TDC in 65 nm FPGA technology," in *Proc. 10th Conf. Ph.D. Res. Microelectron. Electron. (PRIME)*, Grenoble, France, Jun. 2014, pp. 1–5. [Online]. Available: <http://ieeexplore.ieee.org/document/6872727/>
- [11] Z. Yin, S. Liu, X. Hao, S. Gao, and Q. An, "A high-resolution time-to-digital converter based on multi-phase clock implement in field-programmable-gate-array," in *Proc. 18th IEEE-NPSS Real Time Conf.*, Jun. 2012, pp. 1–4.
- [12] K. Katoh and K. Namba, "A low area calibration technique of TDC using variable clock generator for accurate on-line delay measurement," in *Proc. 16th Int. Symp. Qual. Electron. Design*, Santa Clara, CA, USA, Mar. 2015, pp. 430–434. [Online]. Available: <http://ieeexplore.ieee.org/document/7085464/>

error. Finally, we divide the result by $N = 19$, as it is well-known that the width of a pulse in an N -node ring oscillator is N times the node timescale.

- [13] K. Katoh and K. Namba, "Time-to-digital converter-based maximum delay sensor for on-line timing error detection in logic block of very large scale integration circuits," *Sensors Mater.*, vol. 27, no. 10, pp. 933–943, 2015. [Online]. Available: <http://myukkk.org/SM2017/article.php?ss=1158>
- [14] R. Cui and K. Namba, "A calibration technique for DVMC with delay time controllable inverter," *IPSSJ Trans. Syst. LSI Des. Methodol.*, vol. 9, pp. 30–36, Feb. 2016. [Online]. Available: https://www.jstage.jst.go.jp/article/ipsjtsldm/9/0/9_30/_article
- [15] K. Cui, X. Li, and R. Zhu, "A high-linearity time-to-digital converter based on dynamically delay-adjustable looped carry chains on FPGAs," *Rev. Sci. Instrum.*, vol. 89, no. 8, Aug. 2018, Art. no. 084704, doi: [10.1063/1.5038146](https://doi.org/10.1063/1.5038146).
- [16] M.-A. Daigneault and J. P. David, "A high-resolution time-to-digital converter on FPGA using dynamic reconfiguration," *IEEE Trans. Instrum. Meas.*, vol. 60, no. 6, pp. 2070–2079, Jun. 2011.
- [17] H. Chen, Y. Zhang, and D. D.-U. Li, "A low nonlinearity, missing-code free time-to-digital converter based on 28-nm FPGAs with embedded bin-width calibrations," *IEEE Trans. Instrum. Meas.*, vol. 66, no. 7, pp. 1912–1921, Jun. 2017. [Online]. Available: <https://ieeexplore.ieee.org/document/7898491/>
- [18] P. Ghoshal and S. K. Sen, "A bit swap logic (BSL) based bubble error correction (BEC) method for flash ADCs," in *Proc. 2nd Int. Conf. Control, Instrum., Energy Commun. (CIEC)*, Kolkata, India, Jan. 2016, pp. 111–115. [Online]. Available: <http://ieeexplore.ieee.org/document/7513760/>
- [19] (2019). *AN 661: Implementing Fractional PLL Reconfiguration With Altera PLL and Altera PLL Reconfig IP Cores*. [Online]. Available: <https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/an/an661.pdf>
- [20] J. Tatsukawa. (2019). *MMCM and PLL Dynamic Reconfiguration*. [Online]. Available: https://www.xilinx.com/support/documentation/application_notes/xapp888_7Series_DynamicRecon.pdf
- [21] *Factory-Programmable Any-Frequency CMOS Clock Generator + PLL*, Silicon Lab., Austin, TX, USA, 2015, p. 6. [Online]. Available: <https://www.semiconductors.com/pages/asp/DownloadDirect.asp?sid=1635347577853>
- [22] (2005). *LCELL WYSIWYG Description for the Cyclone II Architecture*. [Online]. Available: <https://community.intel.com/t5/Intel-Quartus-Prime-Software/Designing-with-low-level-primitives/tid-p110485?profile.language=en&attachment-id=30610>
- [23] H. Huang, "A 0.1 ps resolution coarse-fine time-to-digital converter with 2.21 ps single-shot precision," Ph.D. dissertation, Univ. Texas Dallas, Richardson, TX, USA, May 2018, p. 8. [Online]. Available: <https://utd-ir.tdl.org/bitstream/handle/10735.1/5885/ETD-5608-013-HUANG-7898.16.pdf?sequence=5&isAllowed=y>
- [24] M. Ghil and A. Mullhaupt, "Boolean delay equations. II. Periodic and aperiodic solutions," *J. Stat. Phys.*, vol. 41, nos. 1–2, pp. 125–173, Oct. 1985. [Online]. Available: <http://link.springer.com/10.1007/BF01020607>
- [25] J. Lohmann, O. D'Huys, N. D. Haynes, E. Schöll, and D. J. Gauthier, "Transient dynamics and their control in time-delay autonomous Boolean ring networks," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 95, no. 2, Feb. 2017, Art. no. 022211, doi: [10.1103/PhysRevE.95.022211](https://doi.org/10.1103/PhysRevE.95.022211).
- [26] D. P. Rosin, *Autonomous Boolean Networks on Electronic Chips* (Springer Theses). Cham, Switzerland: Springer, 2015, ch. 4, p. 37. [Online]. Available: http://link.springer.com/10.1007/978-3-319-13578-6_3
- [27] N. Charlot, D. Canaday, A. Pomerance, and D. J. Gauthier, "Hybrid Boolean networks as physically unclonable functions," *IEEE Access*, vol. 9, pp. 44855–44867, 2021.
- [28] H. Li, C. Wang, S. An, X. Lu, Y. Dong, S. Liu, H. Baghaei, Y. Zhang, R. Ramirez, and W.-H. Wong, "An accurate timing alignment method with time-to-digital converter linearity calibration for high-resolution TOF PET," *IEEE Trans. Nucl. Sci.*, vol. 62, no. 3, pp. 799–804, Jun. 2015. [Online]. Available: <http://ieeexplore.ieee.org/document/7117473/>
- [29] Y.-H. Chen, "Time resolution improvement using dual delay lines for field-programmable-gate-array-based time-to-digital converters with real-time calibration," *Appl. Sci.*, vol. 9, no. 1, p. 20, Dec. 2018. [Online]. Available: <http://www.mdpi.com/2076-3417/9/1/20>
- [30] J. Y. Won and J. S. Lee, "Time-to-digital converter using a tuned-delay line evaluated in 28-, 40-, and 45-nm FPGAs," *IEEE Trans. Instrum. Meas.*, vol. 65, no. 7, pp. 1678–1689, Jul. 2016. [Online]. Available: <http://ieeexplore.ieee.org/document/7448918/>



NOELOIKEAU F. CHARLOT received the B.S. degree in physics and biological engineering from the University of Hawai'i at Mānoa, Honolulu, HI, USA, in 2017, and the M.S. degree in physics from The Ohio State University, Columbus, OH, USA, in 2020, where he is currently pursuing the Ph.D. degree in physics.

He is also a Research Assistant with the QuantInfo Lab and CYAN cybersecurity collaboration at OSU. Previously, he interned in the Gravitational Astrophysics Laboratory at NASA Goddard and the Dark Matter Detection Laboratory at UH Manoa. He is a prior McNair Scholar. His industrial experience includes bioreactor design and photonics at ProtaCulture LLC. His research interests include network science, artificial intelligence, ultrafast electronics, and quantum gravity.



DANIEL J. GAUTHIER (Member, IEEE) received the B.S., M.S., and Ph.D. degrees from the University of Rochester, Rochester, NY, USA, in 1982, 1983, and 1989, respectively. His Ph.D. research on "Instabilities and chaos of laser beams propagating through nonlinear optical media" was supervised by Prof. R. W. Boyd and supported in part through a University Research Initiative Fellowship. From 1989 to 1991, he developed the first CW two-photon optical laser as a Postdoctoral

Research Associate under the mentorship of Prof. T. W. Mossberg at the University of Oregon. In 1991, he joined the Faculty of Duke University, Durham, NC, USA, as an Assistant Professor of physics and was named as a Young Investigator of the U.S. Army Research Office, in 1992, and the National Science Foundation, in 1993. He was a Robert C. Richardson Professor of physics at Duke, from 2011 to 2015, the Chair of the Duke Physics Department, from 2005 to 2011, the interim chair, in spring 2015, and was a Founding Member of the Duke Fitzpatrick Institute for Photonics. He moved to The Ohio State University, in 2016. He is currently a Professor of physics and electrical and computer engineering with The Ohio State University. His research interests include reservoir computing, synchronization and control of the dynamics of complex networks in electronic and optical systems, quantum communication, and nonlinear quantum optics. He is a fellow of the Optical Society of America and the American Physical Society.



ANDREW POMERANCE was born in Washington, DC, USA, in 1980. He received the B.S. and M.S. degrees in electrical and computer engineering from Carnegie Mellon University, Pittsburgh, PA, USA, in 2002, and the Ph.D. degree in physics from the University of Maryland, College Park, MD, USA, in 2009.

From 2009 to 2013, he was with Raytheon Applied Signal Technology, Tyson's Corner, VA, USA. He is currently the President of Potomac Research LLC, Alexandria, VA, USA. His research interests include nonlinear dynamics with applications to machine learning and cryptography.

...